

Lesson:



Queues - 2



Pre Requisites:

- Queues

List of concepts involved :

- Interview Questions on Queues

Problem 1: Reverse a Queue

Q1. Insert the elements 1,2,3,4 in a queue and reverse them so that the order of the elements in the queue becomes 4,3,2,1.

Explanation:

- We first insert the elements in the queue in the order 1,2,3,4.
- Now, we use an extra stack to reverse the order as stack follows LIFO method.
- Remove the elements one by one from the queue and insert into the stack.
- Remove the elements from the stack and insert back into the queue.
- To confirm it, we print the elements in the queue by removing them.

Code:

<https://pastebin.com/YE9h14H2>

Output:

```
"C:\Program Files\Java\jdk-14.0.1
4 3 2 1
Process finished with exit code 0
|
```

Time Complexity: $O(n)$ as we traverse over all the elements thrice, once while removing from queue, second while removing from stack and third while printing it.

Space Complexity: $O(n)$ due to extra space used by Stack.

Problem 2: Reverse first k elements of a Queue

Q2. Insert the elements 1,2,3,4,5 in a queue and reverse the first 3 numbers so that the order of the elements in the queue becomes 3, 2, 1, 4, 5.

Explanation:

- We first insert the elements in the queue in the order 1,2,3,4, 5.
- Now, we use an extra stack to reverse the order as stack follows LIFO method.
- Remove the first $k = 3$ elements one by one from the queue and insert into the stack.
- Remove the 3 elements from the stack and insert back into the queue.
- Now the order of the queue is 4,5,3,2,1. To get the desired order, we remove $n-k$ elements from the queue = 2 and add them to the queue. So they get removed from the front of the queue and get added to the back in the same order.

- To confirm it, we print the elements in the queue by removing them.

Code:

<https://pastebin.com/HJ3DpPdu>

Output:

```
"C:\Program Files\Java\jdk-14.0.1
3 2 1 4 5
Process finished with exit code 0
```

Time Complexity: $O(k)$

Space Complexity: $O(k)$ as there are k elements using the extra stack space.

Problem-3: Implement Stack using a Queue.

Q3. You need to implement the functioning of a stack(LIFO) using a Queue.

Create separate functions for the major stack operations: push(), peek(), size() and pop().

```
push(1)
push(2)
push(3)
push(4)
push(5)
peek()
push(6)
pop()
peek()
size()
```

Explanation:

- For push function, we simply add the element to the queue.
- For pop function, we remove and add the elements from the queue and into it for size - 1 times. The element at the top of the queue is now the one that we inserted last originally. Simply remove it from the queue and print it.
- For peek function, we remove and add the elements from the queue and into it for size - 1 times. The element at the top of the queue is now the one that we inserted last originally. Remove it, print it and put it back into the queue.
- For size function, simply print size of queue.

Code:

<https://pastebin.com/nEkqrRTX>

Output:

```
"C:\Program Files\Java\jdk-14.0.1
5
6
5
5
Process finished with exit code 0
```

Time Complexity: For push operation: $O(1)$, for pop operation: $O(n)$, for peek operation: $O(n)$, for size operation: $O(1)$

Space Complexity: $O(1)$

Problem-4: Implement Queue using Stacks

Q4. You need to implement the functioning of a queue(FIFO) using Stacks.

Create separate functions for the major queue operations: `add()`, `peek()`, `size()` and `remove()`.

```
add(1)
add(2)
add(3)
add(4)
add(5)
peek()
add(6)
remove()
peek()
size()
```

Explanation:

- We need two stacks.
- There are 2 ways to approach this problem making it either add efficient or remove efficient.
- To make it remove efficient, we add the elements to the queue. For remove or peek operation, we simply pop and push all the elements to the other stack. The element on the top of this stack is the first one inserted in the original stack. Simply print it. Push all elements back to original stack.
- For add efficient approach, while adding a new element in the stack, we pop and push all elements to the other stack and then add the new element to the original stack and then pop and push elements from helper stack to original stack.

Code:

<https://pastebin.com/BNUQh9Sx>

Output:

```
"C:\Program Files\Java\jdk-14.0.1\
1
1
2
5

Process finished with exit code 0
```

Time Complexity: For add operation: $O(1)$, for remove operation: $O(n)$, for peek operation: $O(n)$, for size operation: $O(1)$

Space Complexity: $O(1)$

Problem-5: First Negative in each window of size k

Q5. Given an array of length n and a positive integer k, find the first negative integer for each window(contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window.

Input1:

n = 5
arr[] = {-8, 2, 3, -6, 10}
k = 2

Output1:

-8 0 -6 -6

Explanation:

First negative integer for each window of size k

{-8, 2} = -8

{2, 3} = 0 (does not contain a negative integer)

{3, -6} = -6

{-6, 10} = -6

Input2:

n = 8
arr[] = {12, -1, -7, 8, -15, 30, 16, 28}
k = 3

Output2:

-1 -1 -7 -15 -15 0

Explanation:

- We create a Queue that stores only useful elements of the current window of k elements.
- An element is useful if it is in the current window and it is a negative integer.
- We process all array elements one by one and maintain queue to contain useful elements of current window and these useful elements are all negative integers.
- For a particular window, if queue is not empty then the element at front of the queue is the first negative integer for that window, else that window does not contain a negative integer.

Code:

<https://pastebin.com/0bYpk55y>

Output:

```
"C:\Program Files\Java\jdk-14.0.1
Enter the length of the array:
5
Enter the elements of the array:
-8 2 3 -6 10
Enter the size of window:
2
-8 0 -6 -6
Process finished with exit code 0
```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

Problem-6: Reorder Queue

Q6. Given the elements in a queue, the number of elements in the queue are even, interweave the first half of the queue with the second half.

Input:

1 2 3 4 5 6 7 8

Output:

1 5 2 6 3 7 4 8

Explanation:

- We use an extra stack space.
- First insert the first half of the queue into the stack to reverse the order of it and now insert them to the rear of the queue.
- Similarly, do it for the other half of the queue but don't insert them into the queue.
- Now, run a loop for half of the queue and one by one first insert the element from the stack to the queue and then remove element from the queue and insert to its rear.
- Now, reverse the queue using the stack.

Code:

<https://pastebin.com/z7RLwFgI>

Output:

```
"C:\Program Files\Java\jdk-14.0.1"
1 5 2 6 3 7 4 8
Process finished with exit code 0
```

Upcoming lecture

- Collections Framework